DEPARTMENT OF COMPUTER SCIENCE

RECORD NOTE

Record work submitted to Shri Nehru Maha Vidyalaya College of Arts and Science(Autonomous), affiliated to Bharathiar University, Coimbatore in partial fulfillment of the requirement for the Degree of

Master of Science in Computer Science

PRACTICAL IV - Advanced Java Lab

SHRI NEHRU MAHA VIDYALAYA COLLEGE OF ARTS & SCIENCE (AUTONOMOUS)

(Affiliated to Bharathiar University)

Shri Gambhirmal Bafna Nagar

Malumachampatti, Coimbatore-641050

MARCH – 2026

SHRI NEHRU MAHA VIDYALAYA
COLLEGE OF ARTS AND SCIENCE
(SNMV)
AUTONOMOUS
(Affiliated to Bharathiar University, Coimbatore, Re-accredited with "A+" Grade by NAAC)
Shri Gambhirmal Bafna Nagar, Malumachampatti, Coimbatore - 641 050. Tamil Nadu, India.
Estd. 1989
Estd. 1964

MASTER OF SCIENCE IN COMPUTER SCIENCE

This is to certify that it is a bonafide record work done by

_____ Studying I year M.Sc . Computer Science

Reg .No. _____

_____ _____

Staff In – Charge **Head of the Department**

Submitted for the Autonomous Practical Examination

held on _____

_____ _____

INTERNAL EXAMINER EXTERNAL EXAMINER

CONTENT

S.NO	DATE	TITLE	PAGE NO.
1.		Welcome Servlet	
2.		Servlet Purchase Order	
3.		JSP Student Marksheet	
4.		JSP Purchase Order	
5.		JSP Pay Slip	
6.		JDBC Database Connectivity	
7.		Servlet Using Form Data	
8.		Table Of Headers	
9.		JSP Session Object	
10.		RMI Client Server Application	
11.		Calculator Application	
12.		Socket Programming	

Staff In-Charge

EX.NO: 1	**Welcome Servlet**
DATE:	

AIM:

To display a welcome message using Servlet.

ALGORITHM:

Step1: Start the application by deploying the servlet project on the web server (e.g.ApacheTomcat).

Step2: Server reads web.xml file and identifies the servlet configuration**:**

- Servlet name: HelloWorld
- Servlet class: HelloWorld
- URL pattern: /servlets/servlet/HelloWorld

Step3: Client sends a request through a browser using the mapped URL:

http://localhost:8080/projectname/servlets/servlet/HelloWorld

Step4: Web container receives the request and matches the URL pattern with the servlet mapping defined in web.xml.

Step5: Container loads the HelloWorld servlet class and creates its object.

Step6: Servlet container calls the service() method, passing:

- HttpServletRequest object
- HttpServletResponse object

Step7: Inside the service() method:

- PrintWriter object is obtained from response

- The message "welcome servlet" is written to the response output stream

Step8: Server sends the response back to the client browser, and the message welcome servlet is displayed on the webpage.

PROGRAM:

web.xml:

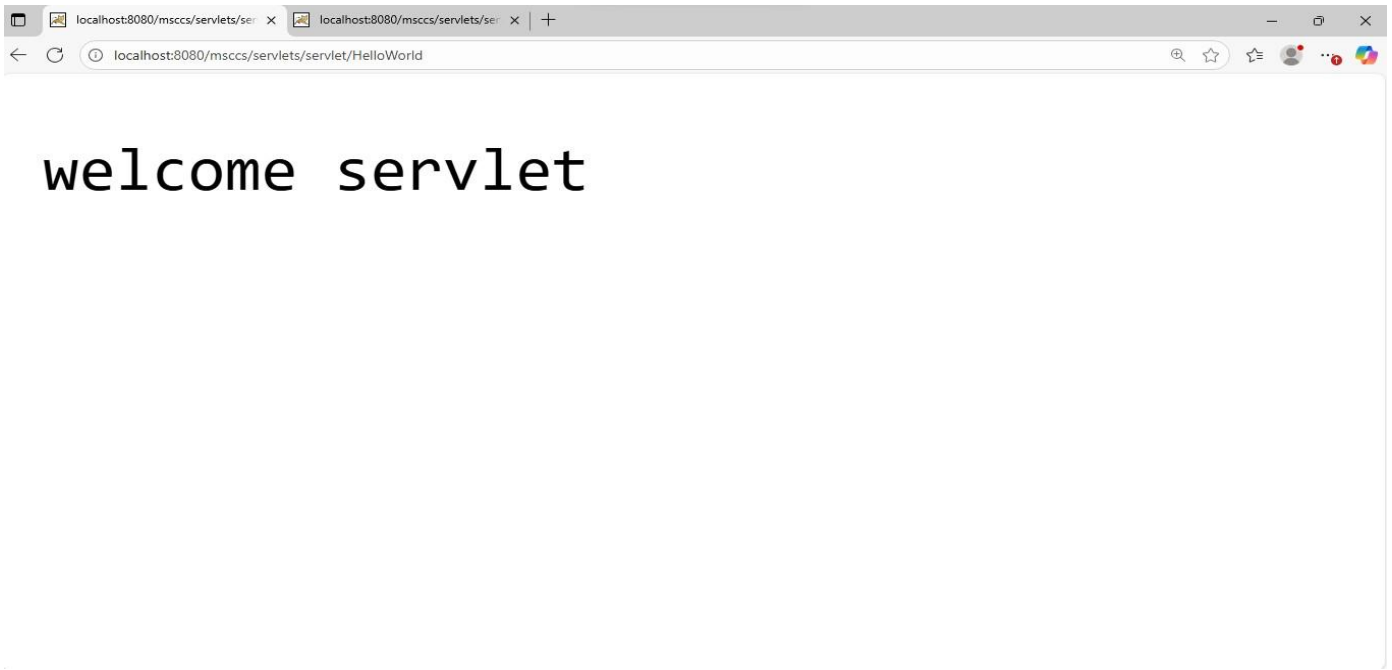
```
<web-app>
    <servlet>
        <servlet-name>HelloWorld</servlet-name>
        <servlet-class>HelloWorld</servlet-class>
    </servlet>
    <servlet-mapping>
        <servlet-name>HelloWorld</servlet-name>
        <url-pattern>/servlets/servlet/HelloWorld</url-pattern>
    </servlet-mapping>
</web-app>
```

HelloWorld.java:

```
import java.io.*;
import java.util.*;
import jakarta.servlet.*;
import jakarta.servlet.http.*;
/**
 * The simplest possible servlet.
 *
 * @author James Duncan Davidson
 */
```

```
public class HelloWorld extends HttpServlet {

        public void service(HttpServletRequest request, HttpServletResponse response)

         throws  IOException, ServletException {

                 PrintWriter out = response.getWriter();

                out.println("welcome servlet");

    }

}
```

OUTPUT:



RESULT:

Thus the above program was executed and the output is verified successfully.

EX.NO: 2	**Servlet Purchase Order**
DATE:	

AIM:

To design a purchase order form using Html form and servlet.

ALGORITHM:

Step1:Start the process.

Step2: Load the HTML page (purchase.html) in the browser.

Step3: User enters customer details:

1. Customer Name
2. Address
3. Mobile Number
4. Selects an item from the dropdown
5. Enters number of items

Step4: User clicks the Submit button.

Step5: The form data is sent using POST method to the URL:

/msccs/servlets/servlet/purchaseservlet.

Step6: The web.xml file maps the URL pattern to the purchaseservlet class.

Step7: The servlet container (e.g., Apache Tomcat) loads and executes the purchaseservlet.

Step8: The doPost() method is invoked, and the servlet:

- Retrieves form data using request.getParameter().

Step9: The servlet:

- Sets response content type to "text/html".
- Uses PrintWriter to display customer details and order information.

Step10:The generated HTML response is sent back to the browser and displayed to the user.

PROGRAM:

Web.xml:

```
<web-app>

    <servlet>
        <servlet-name>purchaseservlet</servlet-name>
        <servlet-class>purchaseservlet</servlet-class>
    </servlet>

    <servlet-mapping>
        <servlet-name>purchaseservlet</servlet-name>
        <url-pattern>/servlets/servlet/purchaseservlet</url-pattern>
    </servlet-mapping>

</web-app>
```

Purchaseservlet.java:

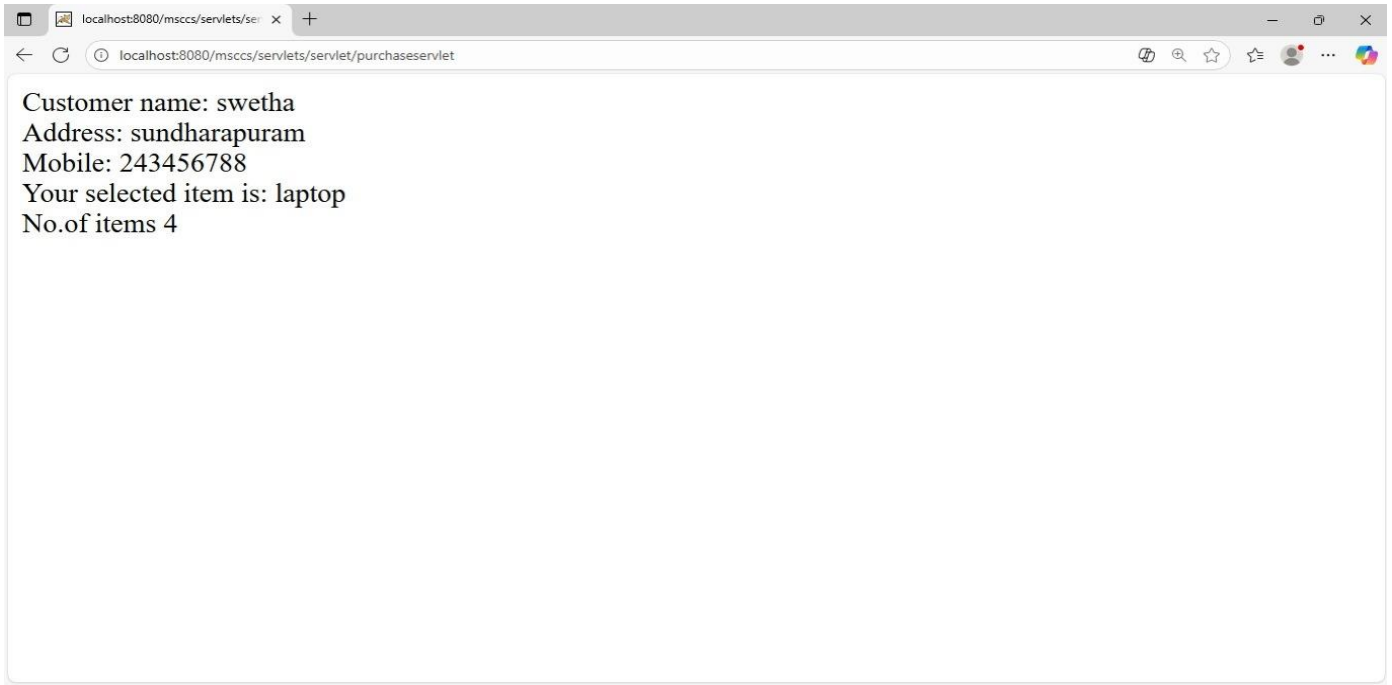
```
import java.io.*;
import jakarta.servlet.*;
import jakarta.servlet.http.*;

public class purchaseservlet extends HttpServlet {

public void doPost(HttpServletRequest request,HttpServletResponse response)
            throws ServletException,IOException{
   String cname=request.getParameter("cname");
   String addr=request.getParameter("add");
   String mob=request.getParameter("mno");
   String items=request.getParameter("items");
   String noi=request.getParameter("noi");
   String email=request.getParameter("email");
   response.setContentType("text/html");
   PrintWriter pw = response.getWriter();
   pw.println("Customer name: ");
   pw.println(cname);
   pw.println("<br>");
   pw.println("Address: ");
```

```
   pw.println(addr);
   pw.println("<br>");
   pw.println("Mobile: ");
   pw.println(mob);
   pw.println("<br>");
   pw.println("Your selected item is: ");
   pw.println(items);
   pw.println("<br>");
   pw.println("No.of items ");
   pw.println(noi);
   pw.println("<br>");
   pw.close();
 }
}
```

Purchaseform.html:

```
<html>
<head>
<title>
Purchase Form
</title>
</head>
<body>
<form name="form1" method="post"
action="http://localhost:8080/msccs/servlets/servlet/purchaseservlet">
Customer Name<input type="text" name="cname"><br>
Address:<input type="text" name="add"><br>
Mobile:<input type="text" name="mno"><br>
Items to be ordered:<select name="items">
<option value="laptop">Laptop</option>
<option value="Mouse">Mouse</option>
<option value="keyboard">keyboard</option>
<option value="Pendrive">Pendrive</option>
</select><br>
No.of items:<input type="text" name="noi"><br>
<input type="Submit">
</form>
</body>
</html>
```

OUTPUT:



RESULT :

Thus the above program was executed and the output is verified successfully.

EX.NO: 3	**JSP Student Marksheet**
DATE:	

AIM:

To develop a program for calculating the percentage of marks of a student using JSP.

ALGORITHM:

Step1:Start the program.

Step2:Display an HTML form to enter student details:

- Student Name
- Register Number
- Course
- Marks in Java, AOS, Python, and DM

Step3:Accept the input values from the user and submit the form data to mark.jsp using the POST method.

Step4: Retrieve all submitted values in mark.jsp using request.getParameter().

Step5: Convert the marks (Java, AOS, Python, DM) from String to Integer using Integer.parseInt().

step6: Calculate:

- Total = Java + AOS + Python + DM
- Average = Total / 4

step7: Determine the grade based on the average:

- Above 90 → Distinction
- 80–89 → A Grade

- 70–79 → B Grade
- 60–69 → C Grade
- 50–59 → D Grade

step8: Display the student details, total, average, and grade, then stop the program.

PROGRAM :

Student.html:

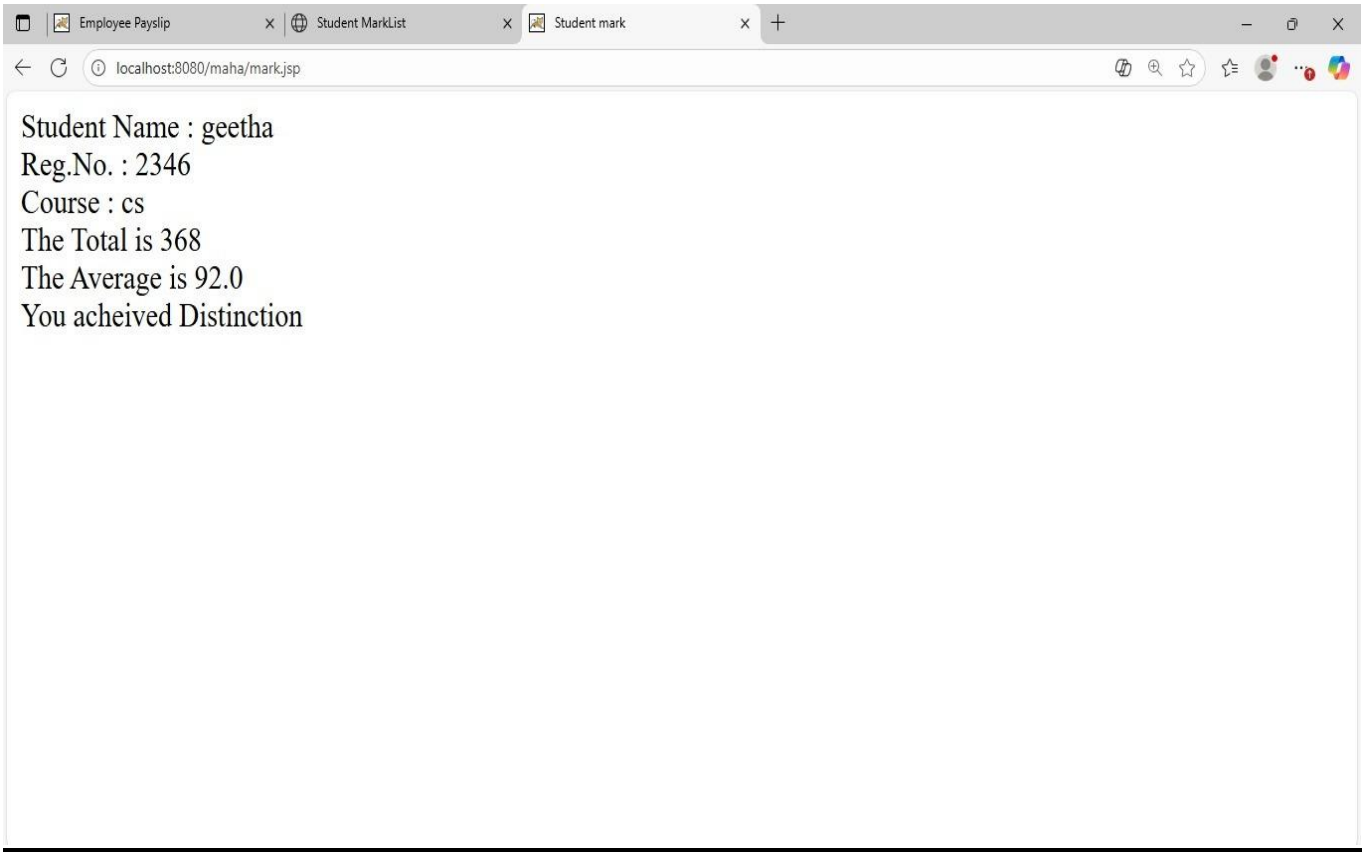
```
<html>
<head>
<title>Student MarkList</title></head>
<body>
<form action="http://localhost:8080/vinodjsp/mark.jsp" method="post">
<h1><center>Student Marklist<center></h1>
Student Name:<input type="text" name="sname"><br>
Register No.:<input type="text" name="rno"><br>
Course:<input type="text" name="cname"><br>
<h3>Marks</h3>
Java:<input type="text" name="java"><br>
Aos:<input type="text" name="aos"><br>
Python:<input type="text" name="python"><br>
Dm:<input type="text" name="dm"><br>
<input type="submit"><br>
</form>
</body>
</html>
```

Mark.jsp:

```
<html>
<head><title>Student mark</title></head>
<body>
<% String sname=request.getParameter("sname");
String rno=request.getParameter("rno");
String course=request.getParameter("cname");
int java=Integer.parseInt(request.getParameter("java"));
int aos=Integer.parseInt(request.getParameter("aos"));
int python=Integer.parseInt(request.getParameter("python"));
int dm=Integer.parseInt(request.getParameter("dm"));
int total=java+aos+python+dm;
out.println("Student Name : "+sname);
out.println("<br>");
out.println("Reg.No. : "+rno);
```

```
out.println("<br>");
out.println("Course : "+course);
out.println("<br>");
out.println("The Total is "+total);
out.println("<br>");
double avg = total/4;
out.println("The Average is "+avg);
out.println("<br>");
if(avg>90)
{
out.println("You acheived  Distinction \n");
}
else if(avg>=80)
{
out.println("You acheived A Grade \n");
}
else if(avg>=70)
{
out.println("You acheived B Grade \n");
}
else if(avg>=60)
{
out.println("You acheived C Grade \n");
}
else if(avg>=50)
{
out.println("You acheived D Grade \n");
}
%>
</body>
</html>
```

OUTPUT:



RESULT:

Thus the above program was executed and the output is verified successfully.

EX.NO: 4	**JSP Purchase Order**
DATE:	

AIM:

To design a purchase order form using Html form and JSP.

ALGORITHM:

Step1: Start the program.

Step2: Display the Purchase Form (HTML page) with input fields:

- Customer Name
- Address
- Mobile
- Email
- Item selection (Laptop, Mouse, Keyboard, Pen drive)
- Number of items

Step3: User enters the required details in the form fields.

Step4: User clicks the Submit button, and the form data is sent using the POST method to purchase.jsp.

Step5: JSP page receives the request and reads the form data using request.getParameter() for:

- cname
- addr
- mob
- mail
- items
- noi

Step6: Store the received values in corresponding string variables.

Step7: Display the entered details on the browser using out.println() statements.

Step8: End the program.

PROGRAM :

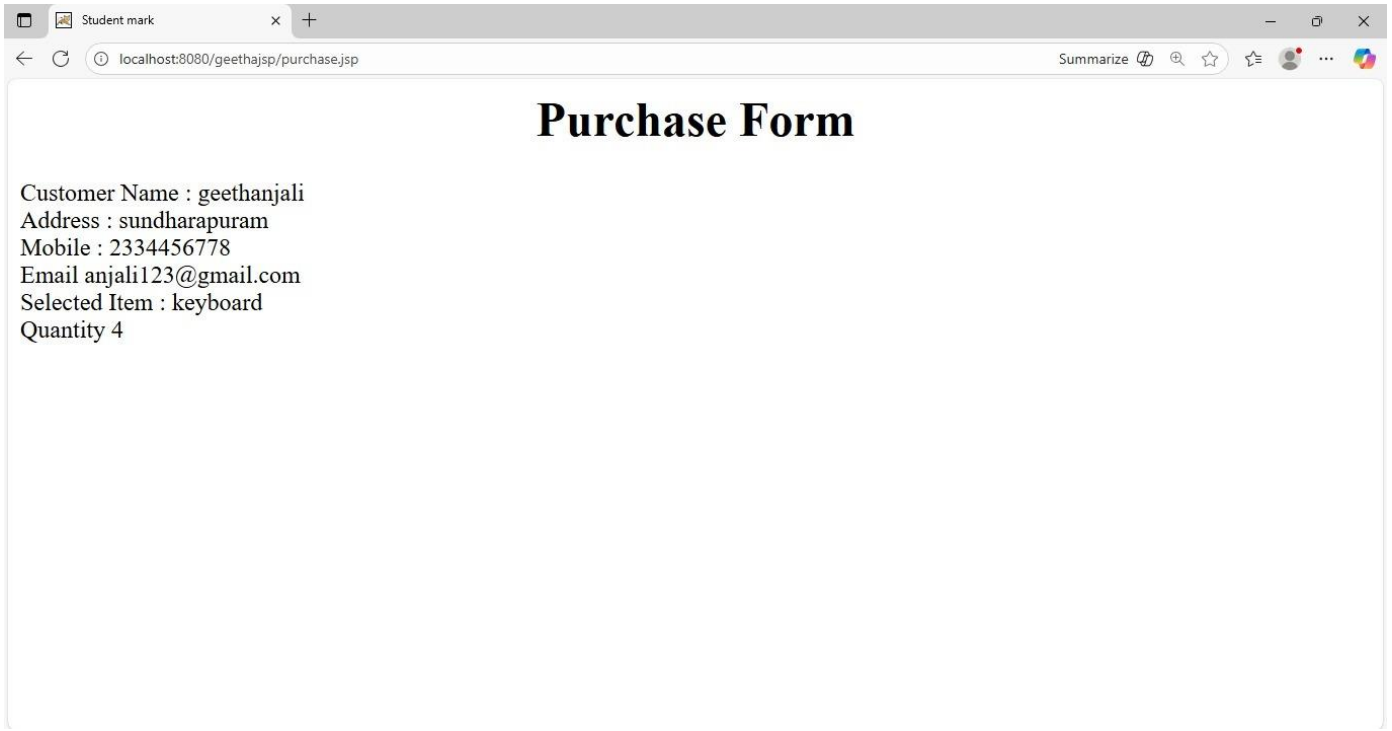
Purchaseform.html:

```
<html>
<head>
<title>
Purchase Form
</title>
</head>
<body>
<h1>Purchase Form</h1>
<form name="form1" method="post" action="http://localhost:8080/vinodjsp/purchase.jsp">
Customer Name:<input type="text" name="cname"><br>
Address:<input type="text" name="addr"><br>
Mobile:<input type="text" name="mob"><br>
Email:<input type="text" name="mail"><br>
Items to be ordered:<select name="items">
<option value="laptop">Laptop</option>
<option value="Mouse">Mouse</option>
<option value="keyboard">keyboard</option>
<option value="Pendrive">Pendrive</option>
</select><br>
No.of items:<input type="text" name="noi"><br>
<input type="Submit">
</form>
</body>
</html>
```

Purchase.jsp:

```
<html>
<head><title>Student mark</title></head>
<body>
<h1><center>Purchase Form</center></h1>
<%
String cname=request.getParameter("cname");
String address=request.getParameter("addr");
String mobile=request.getParameter("mob");
String email=request.getParameter("mail");
String item=request.getParameter("items");
String noi=request.getParameter("noi");
out.println("Customer Name : "+cname);
out.println("<br>");
out.println("Address : "+address);
out.println("<br>");
out.println("Mobile : "+mobile);
out.println("<br>");
out.println("Email "+email);
out.println("<br>");
out.println("Selected Item : "+item);
out.println("<br>");
out.println("Quantity "+noi);
out.println("<br>");

%>
</body>
</html>
```

OUTPUT:



RESULT :

Thus the above program was executed and the output is verified successfully.

EX.NO: 5	**JSP Pay Slip**
DATE:	

AIM:

To prepare a Employee pay slip using JSP.

ALGORITHM:

Step1: Start the program.

Step2: Display an HTML form to the user with input fields:

- Employee Name
- Employee ID
- Department
- Basic Pay
- HRA
- CCA
- PF

Step3:. Accept the user input and submit the form data to payslip.jsp using the POST method.

Step4:. Retrieve all form values in payslip.jsp using request.getParameter().

Step5:. Convert salary-related values (Basic Pay, HRA, CCA, PF) from String to Integer using Integer.parseInt().

Step 6. Calculate:

- Gross Pay = Basic Pay + HRA + CCA
- Net Pay = Gross Pay – PF

Step7: Display the employee details along with Gross Pay and Net Pay on the webpage.

Step8: Stop the program.

PROGRAM:

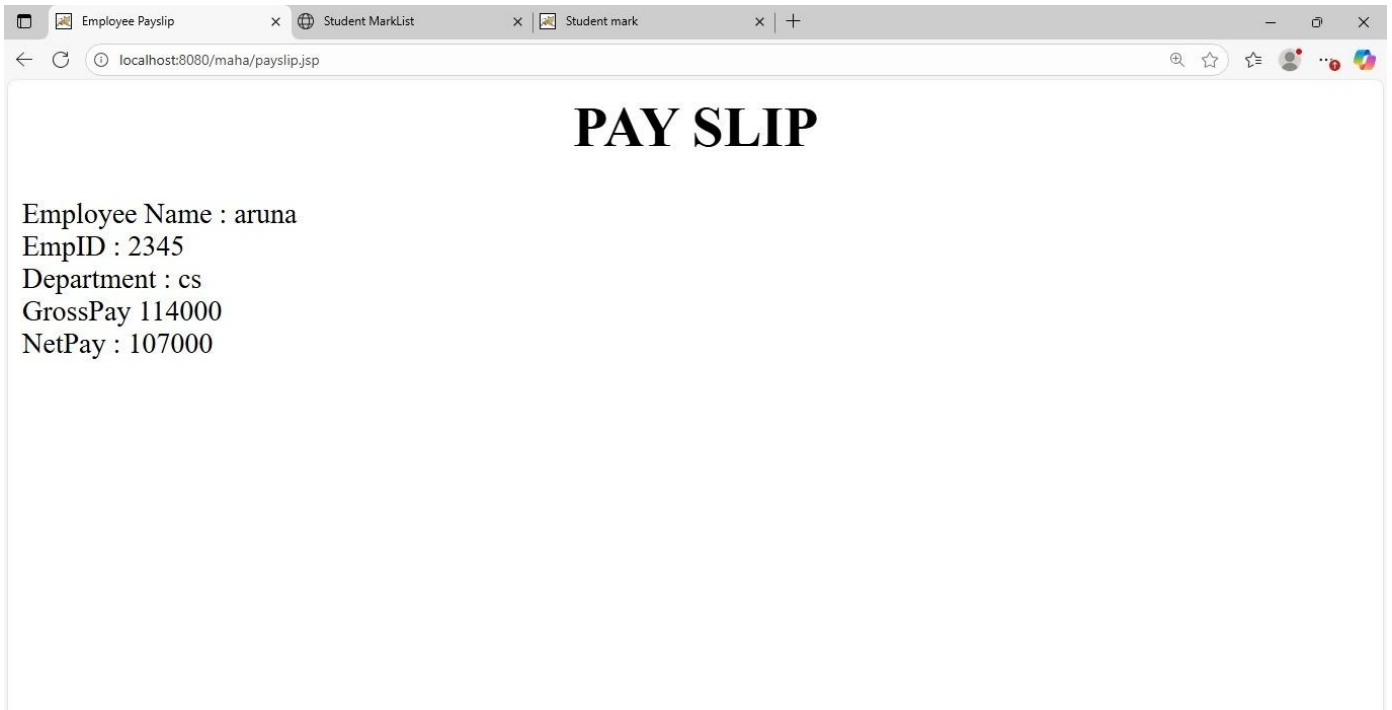
Payslip.html:

```
<html>
<head>
<title>
Pay Slip
</title>
</head>
<body>
<h1>Pay Slip</h1>
<form name="form1" method="post" action="http://localhost:8080/vinodjsp/payslip.jsp">
Employee Name:<input type="text" name="ename"><br>
EmpID:<input type="text" name="eid"><br>
Department:<input type="text" name="dt"><br>
BasicPay:<input type="text" name="bp"><br>
HRA:<input type="text" name="hra"><br>
CCA:<input type="text" name="cca"><br>
PF::<input type="text" name="pf"><br>
<input type="Submit">
</form>
</body>
</html>
```

Payslip.jsp:

```
<html>
<head><title>Employee Payslip</title></head>
<body>
<h1><center>PAY SLIP</center></h1>
<%
String ename=request.getParameter("ename");
String eid=request.getParameter("eid");
String dept=request.getParameter("dt");
int bp=Integer.parseInt(request.getParameter("bp"));
int hra=Integer.parseInt(request.getParameter("hra"));
int cca=Integer.parseInt(request.getParameter("cca"));
int pf=Integer.parseInt(request.getParameter("pf"));
int grosspay = bp+hra+cca;
int netpay = grosspay-pf;
out.println("Employee Name : "+ename);
out.println("<br>");
out.println("EmpID : "+eid);
out.println("<br>");
out.println("Department : "+dept);
out.println("<br>");
out.println("GrossPay "+grosspay);
out.println("<br>");
out.println("NetPay : "+netpay);
out.println("<br>");
%>
</body>
</html>
```

OUTPUT:



RESULT :

Thus the above program was executed and the output is verified successfully.

EX.NO: 6	**JDBC Database Connectivity**
DATE:	

AIM:

To Write a program using JDBC for creating a table, Inserting, Deleting records and list out the records

ALGORITHM:

Step 1: Start the program Initialize database connection details
(URL, USER, PASSWORD) and create a Scanner object for user input.

Step 2: Display Menu and Read Option
Display the options:

1. Create Table
2. Insert Record
3. Delete Record
4. Show Records

Read the user's choice (opt).

Step 3: Establish Database Connection Connect to the MySQL database
using DriverManager.getConnection() and create aStatement object.

Step 4: Perform Selected Operation

- If option = 1 → Execute SQL query to create table REGISTRATION.
- If option = 2 → Read ID and Name from user and insert record into table.
- If option = 3 → Read ID and delete corresponding record.
- If option = 4 → Retrieve and display all records from table.
- Otherwise → Display "Invalid option".

Step 5: Handle Exceptions If any SQL error occurs, display the error message.

Step 6: End the program Close database connection automatically (try-with-resources)
and terminate the program.

PROGRAM :

```
import java.sql.*;
import java.util.*;

class jdb {

    static final String URL = "jdbc:mysql://localhost:3306/msc1";
    static final String USER = "root";
    static final String PASS = "Msccs@123";

 public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.println("1. CREATE TABLE");
        System.out.println("2. INSERT RECORD");
        System.out.println("3. DELETE RECORD");
        System.out.println("4. SHOW RECORDS");
        System.out.print("Enter option: ");

        int opt = sc.nextInt();

        try (Connection conn = DriverManager.getConnection(URL, USER, PASS);
            Statement stmt = conn.createStatement()) {

           switch (opt) {
              case 1:  // CREATE TABLE
                 executeUpdate(stmt, "CREATE TABLE IF NOT EXISTS REGISTRATION
(id INT PRIMARY KEY, name VARCHAR(255))");
                 System.out.println("Table created");
                 break;

              case 2:  // INSERT
                 System.out.print("Enter ID: ");
                 int id = sc.nextInt();
                 sc.nextLine();  // clear buffer
                 System.out.print("Enter Name: ");
                 String name = sc.nextLine();
                 executeUpdate(stmt, "INSERT INTO REGISTRATION VALUES (" + id + ",
'" + name + "')");
                 System.out.println("Record inserted");
                 break;
```

```
case 3:  // DELETE
                System.out.print("Enter ID to delete: ");
                int del = sc.nextInt();
                executeUpdate(stmt, "DELETE FROM REGISTRATION WHERE id=" + del);
                System.out.println("Record deleted");
                break;

            case 4:  // SHOW RECORDS
                ResultSet rs = stmt.executeQuery("SELECT * FROM REGISTRATION");
                if (!rs.next()) {  // If there are no records
                    System.out.println("No records found.");
                } else {
                    System.out.println("ID   NAME");
                    System.out.println("------------");
                    do {
                        System.out.println(rs.getInt(1) + "   " + rs.getString(2));
                    } while (rs.next());
                }
                break;

            default:
                System.out.println("Invalid option");
        }

    } catch (SQLException e) {
        System.out.println("Error: " + e.getMessage());
    }
  }

  // Helper method to execute SQL update queries
  private static void executeUpdate(Statement stmt, String query) throws SQLException {
    stmt.executeUpdate(query);
  }
}
```

OUTPUT:

```
Microsoft Windows [Version 10.0.26200.7705]
(c) Microsoft Corporation. All rights reserved.

D:\jd>set CLASSPATH=.;mysql-connector-j-9.6.0.jar

D:\jd>javac jdb.java

D:\jd>java jdb
1. CREATE TABLE
2. INSERT RECORD
3. DELETE RECORD
4. SHOW RECORDS
Enter option: 1
Table created

D:\jd>java jdb
1. CREATE TABLE
2. INSERT RECORD
3. DELETE RECORD
4. SHOW RECORDS
Enter option: 2
Enter ID: 100
Enter Name: ajith
Record inserted

D:\jd>java jdb
1. CREATE TABLE
2. INSERT RECORD
3. DELETE RECORD
4. SHOW RECORDS
Enter option: 4
ID   NAME
------------
100   ajith
```

```
C:\Windows\System32\cmd.e

3. DELETE RECORD
4. SHOW RECORDS
Enter option: 1
Table created

D:\jd>java jdb
1. CREATE TABLE
2. INSERT RECORD
3. DELETE RECORD
4. SHOW RECORDS
Enter option: 2
Enter ID: 100
Enter Name: ajith
Record inserted

D:\jd>java jdb
1. CREATE TABLE
2. INSERT RECORD
3. DELETE RECORD
4. SHOW RECORDS
Enter option: 4
ID   NAME
------------
100   ajith

D:\jd>java jdb
1. CREATE TABLE
2. INSERT RECORD
3. DELETE RECORD
4. SHOW RECORDS
Enter option: 3
Enter ID to delete: 100
Record deleted

D:\jd>
```

```
C:\Windows\System32\cmd.e

Microsoft Windows [Version 10.0.26200.7705]
(c) Microsoft Corporation. All rights reserved.

C:\Users\Exam 10\mysql-9.6.0-winx64\bin>mysql -u root -P 3307
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 9
Server version: 9.6.0 MySQL Community Server - GPL

Copyright (c) 2000, 2026, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> show databases;
+--------------------+
| Database           |
+--------------------+
| information_schema |
| msc                |
| mysql              |
| performance_schema |
| sys                |
+--------------------+
5 rows in set (0.047 sec)
```

```
mysql> use msc;
Database changed
mysql> show tables;
+---------------+
| Tables_in_msc |
+---------------+
| registration  |
+---------------+
1 row in set (0.020 sec)

mysql> select * from registration;
+-----+-------+
| id  | name  |
+-----+-------+
| 100 | ajith |
+-----+-------+
1 row in set (0.005 sec)

mysql> select * from registration;
Empty set (0.004 sec)

mysql>
```

RESULT :

Thus the above program was executed and the output is verified successfully.

EX.NO: 7	**Servlet Using Form Data**
DATE:	

AIM:

To write a program using java servlet to handle form data.

ALGORITHM:

Step1: Start the web application and deploy it on the servlet container (e.g., Apache Tomcat).

Step2: The user opens color.html in a web browser.

Step3: The HTML page displays a form containing a dropdown list with three color options: Red, Green, and Blue.

Step4: The user selects one color from the dropdown list.

Step5: The user clicks the Submit button.

Step6: The browser sends an HTTP request to the URL /vinod/servlet/colorservlet with the selected color as a request parameter.

Step7: The servlet container reads web.xml, identifies the URL pattern, and maps the request to the colorservlet class.

Step8: The doGet() method of colorservlet is invoked automatically.

Step9: The servlet retrieves the selected color using

request.getParameter("color"), sets the response content type to text/html, and writes the message"The selected color is: <selected color>" to the response.

Step10: The browser receives the response and displays the selected color on the webpage.

PROGRAM:

Color.html:

```
<html>
<body>
<center>
<form name="form1"
     action="http://localhost:8080/msccs/servlets/servlet/colorservlet">
<B>Color:</B>
<select name="color" size="1">
<option value="red">Red</option>
<option value="green">Green</option>
<option value="blue">Blue</option>
</select>
<br><br>
<input type=submit value="Submit">
</form>
</body>
</html>
```

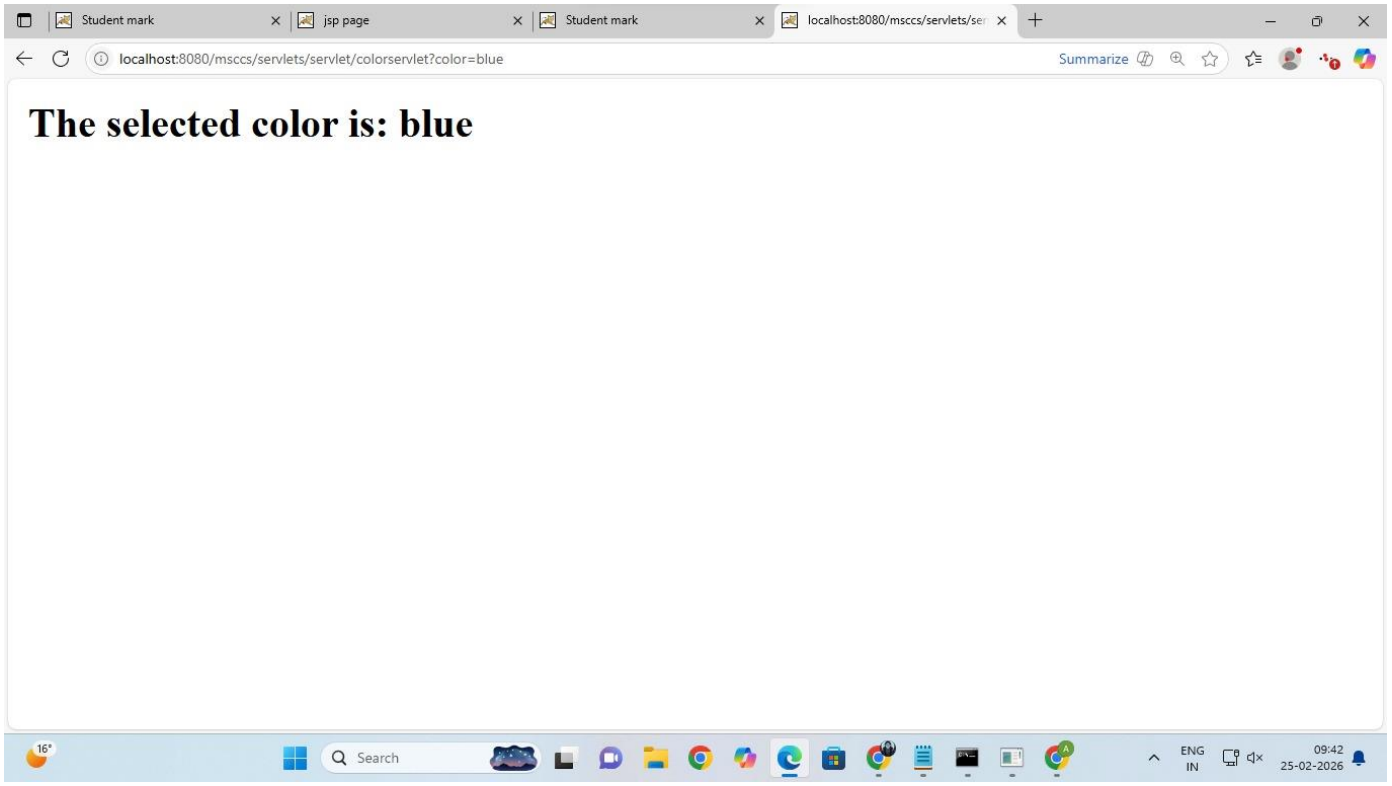
Colorservlet.java:

```
import java.io.*;
import jakarta.servlet.*;
import jakarta.servlet.http.*;

public class colorservlet extends HttpServlet {

public void doGet(HttpServletRequest request,HttpServletResponse response)
          throws ServletException,IOException{
   String color=request.getParameter("color");
   response.setContentType("text/html");
   PrintWriter pw = response.getWriter();
   pw.println("<B>The selected color is: ");
   pw.println(color);
   pw.close();
 }
}
```

Web.xml:

```
<web-app>

    <servlet>
        <servlet-name>colorservlet</servlet-name>
        <servlet-class>colorservlet</servlet-class>
    </servlet>

    <servlet-mapping>
        <servlet-name>colorservlet</servlet-name>
        <url-pattern>/servlets/servlet/colorservlet</url-pattern>
    </servlet-mapping>

</web-app>
```

OUTPUT :



RESULT :

Thus the above program was executed and the output is verified successfully.

EX.NO: 8	**Table Of Headers**
DATE:	

AIM:

To write a simple servlet program to create a table of all the headers it receives along with their associated values.

ALGORITHM:

Step1: Start the program.

Step2: Configure the servlet in web.xml by:

- Defining the servlet name (headers)
- Specifying the servlet class (headers)
- Mapping the URL pattern (/servlets/servlet/headers) to the servlet.

Step3: Deploy the web application on the server and wait for a client request.

Step4: Receive an HTTP request when the user accesses the mapped URL.

Step5: Invoke the service() method of the headers servlet.

Step6: Set the response content type to "text/html" and obtain a

PrintWriter object to send output to the browser.

Step7: Retrieve and display all HTTP header names and their corresponding values:

- Get header names using request.getHeaderNames()
- Iterate through each header
- Fetch header value using request.getHeader()
- Display them in an HTML table format.

Step8: Send the generated HTML response to the client and end the program.

PROGRAM :

Web.xml:

```
<web-app>
      <servlet>
       <servlet-name>headers</servlet-name>
       <servlet-class>headers</servlet-class>
       </servlet>
      <servlet-mapping>
       <servlet-name>headers</servlet-name>
       <url-pattern>/servlets/servlet/headers</url-pattern>
   </servlet-mapping>
 </web-app>
```

Headers.java:

```
import java.io.*;
import java.util.*;
import jakarta.servlet.*;
import jakarta.servlet.http.*;
public class headers extends HttpServlet {
public void service(HttpServletRequest request, HttpServletResponse response) throws
IOException, ServletException {
      response.setContentType("text/html");
      PrintWriter out = response.getWriter();
      out.println("<html><body>");
      out.println("<table border=3><tr><th>Header</th><th>Value</th></tr>");
      Enumeration<String> headerNames = request.getHeaderNames();
      while(headerNames.hasMoreElements())
      {
        String headerName = headerNames.nextElement();
        String headerValue = request.getHeader(headerName);
        out.println("<tr><td>"+headerName+"</td><td>"+headerValue+"</td></tr>");
      }
      out.println("</table></body></html>");
}
}
```

OUTPUT :

Header	Value
Host	localhost:8080
Connection	keep-alive
sec-ch-ua	"Not(A:Brand";v="8", "Chromium";v="144", "Microsoft Edge";v="144"
sec-ch-ua-mobile	?0
sec-ch-ua-platform	"Windows"
Upgrade-Insecure-Requests	1
User-Agent	Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/144.0.0.0 Safari/537.36 Edg/144.0.0.0
Accept	text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
Sec-Fetch-Site	none
Sec-Fetch-Mode	navigate
Sec-Fetch-User	?1
Sec-Fetch-Dest	document
Accept-Encoding	gzip, deflate, br, zstd
Accept-Language	en-US,en;q=0.9

RESULT:

Thus the above program was executed and the output is verified successfully.

EX.NO: 9	**JSP Session Object**
DATE:	

AIM:

To write a program in JSP by using session object

ALGORITHM:

Step1: Start the program.

Step2: Display index.html containing a login form with fields:

- Username
- Password

Step3: User enters username and password and clicks Login.

Step4: Send the entered data to firstpage.jsp using POST method.

Step5: In firstpage.jsp, retrieve the entered username and password using

request.getParameter().

Step6: Compare the entered values with the predefined credentials:

- Username = "vinod"
- Password = "kumar"

Step7: If credentials are correct:

- Create a session.
- Store the username in session using

  session.setAttribute("username", name).
- Redirect to secondpage.jsp.

Step8: If credentials are incorrect:

- Redirect back to index.html?error=1.
- Display alert message "Username or password is wrong".

Step9: In secondpage.jsp:

- Check if session attribute "username" exists.
- If session exists → Display welcome message and Logout button.
- If session does not exist → Redirect to login page (index.html).

Step10: When user clicks Logout:

- In logout.jsp, invalidate the session using session.invalidate().
- Redirect to index.html.
- Stop the program.

PROGRAM :

Index.html:

```
<!DOCTYPE html>
<html>
<head>
    <title>Session</title>

    <script>
        if (window.location.search.includes("error=1")) {
            alert("Username or password is wrong");
        }
    </script>
</head>
<body>
<form action="http://localhost:8080/vgb/firstpage.jsp" method="post">
    Username <input type="text" name="name" required><br><br>
    Password <input type="password" name="pwd" required><br><br>
    <input type="submit" value="Login">
</form>
</body>
</html>
```

Firstpage.jsp:

```
<%@ page language="java" contentType="text/html; charset=UTF-8" %>
<%
String name = request.getParameter("name");
String password = request.getParameter("pwd");

if ("admin".equals(name) && "admin".equals(password)) {
    session.setAttribute("username", name);
    response.sendRedirect("secondpage.jsp");
} else {
    response.sendRedirect("index.html?error=1");
}
%>
```

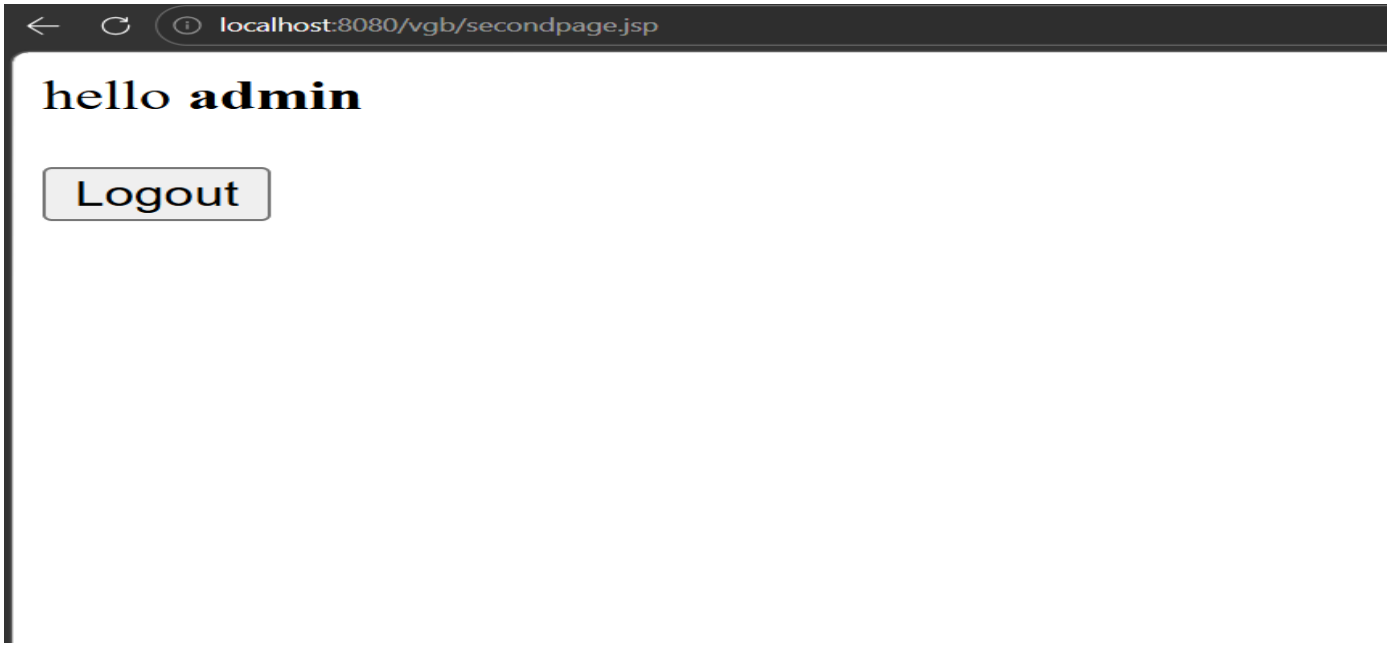
Secondpage.jsp:

```
<%@ page language="java" contentType="text/html; charset=UTF-8"
pageEncoding="UTF-8"%>
<%
String user = (String) session.getAttribute("username");
if (user == null) {
    response.sendRedirect("index.html");
    return;
}
%>

<html>
<head>
<title>jsp page</title>
</head>
<body>

hello <b><%= user %></b><br><br>

<form action="logout.jsp" method="post">
    <input type="submit" value="Logout">
</form>

</body>
</html>
```

Logout.jsp:

```
<%
session.invalidate();   // destroy session
response.sendRedirect("index.html");
%>
```

OUTPUT :



RESULT :

Thus the above program was executed and the output is verified successfully.

EX.NO: 10	**RMI Client Server Application**
DATE:	

AIM:

To write a program to build a simple client Server application using RMI.

ALGORITHM:

Step1: Define the remote interface (addserverintf) that extends Remote and declares the method add(d1, d2) which throws RemoteException.

Step2: Implement the remote interface in addserverimpl by extending Unicast RemoteObject and providing the implementation of the add() method to return the sum of two numbers.

Step3: Create the server program (addserver) that:

- Creates an object of addserverimpl.
- Registers (binds) the remote object to the RMI registry Using Naming.rebind("addserver", asi).

Step4: Start the RMI registry using the command:

rmiregistry

Step5: Run the server program, which binds the remote object in the registry and waits for client requests.

Step6: Create the client program (addclient) that:

- Forms the RMI URL (rmi://hostname/addserver).
- Looks up the remote object using Naming.lookup().

Step7: Client reads two numbers from command-line arguments, converts them into double, and invokes the remote method add(d1, d2).

Step8: Server executes the remote method, returns the sum to the client, and the client displays the result.

PROGRAM :

addclient.java:

```
import java.rmi.*;
public class addclient
{
public static void main(String args[])
{
try
{
String addserverurl = "rmi://" + args[0] + "/addserver";
addserverintf asi1 = (addserverintf)Naming.lookup(addserverurl);
System.out.println("The first number is: "+args[1]);
double d1 = Double.valueOf(args[1]).doubleValue();
System.out.println("The second number is: "+args[2]);
double d2 = Double.valueOf(args[2]).doubleValue();
System.out.println("The sum is: "+asi1.add(d1,d2));
}
catch(Exception e){
        System.out.println("Exception: " +e);
}
}
}
```

addserver.java:

```
import java.net.*;
import java.rmi.*;
public class addserver
{
public static void main(String args[])
{
try
{
addserverimpl asi=new addserverimpl();
Naming.rebind("addserver",asi);
}
catch(Exception e)
{
System.out.println("Exception: "+e);
}
}
}
```

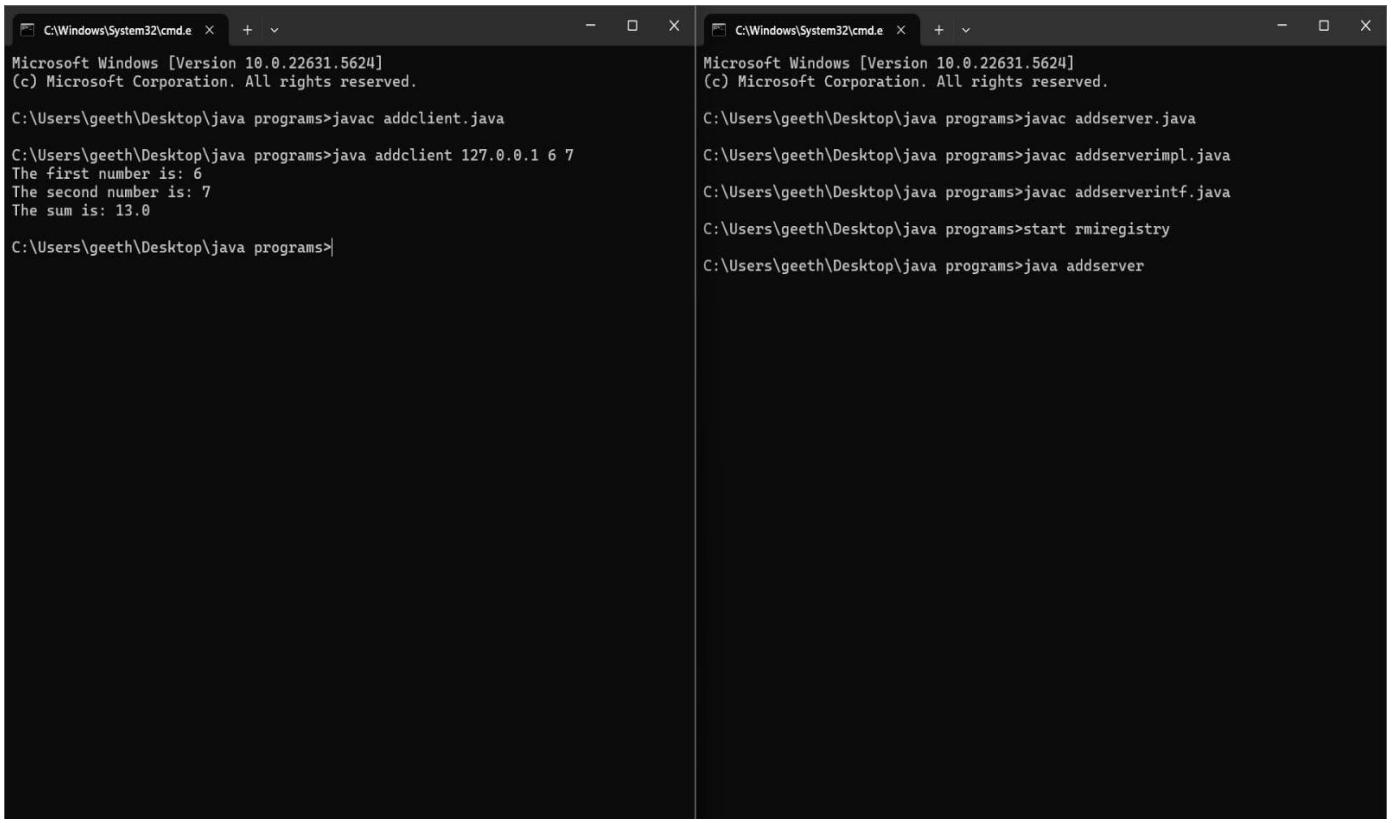
addserverimpl.java:

```
import java.rmi.*;
import java.rmi.server.*;
public class addserverimpl extends UnicastRemoteObject
implements addserverintf{

public addserverimpl() throws RemoteException
{}
public double add(double d1,double d2)throws RemoteException
{
return d1+d2;
}
}
```

addserverintf.java:

```
import java.rmi.*;
public interface addserverintf extends Remote
{
 double add(double d1,double d2) throws RemoteException;
}
```

OUTPUT:



```
C:\Windows\System32\cmd.e
Microsoft Windows [Version 10.0.22631.5624]
(c) Microsoft Corporation. All rights reserved.

C:\Users\geeth\Desktop\java programs>javac addclient.java

C:\Users\geeth\Desktop\java programs>java addclient 127.0.0.1 6 7
The first number is: 6
The second number is: 7
The sum is: 13.0

C:\Users\geeth\Desktop\java programs>
```

```
C:\Windows\System32\cmd.e
Microsoft Windows [Version 10.0.22631.5624]
(c) Microsoft Corporation. All rights reserved.

C:\Users\geeth\Desktop\java programs>javac addserver.java

C:\Users\geeth\Desktop\java programs>javac addserverimpl.java

C:\Users\geeth\Desktop\java programs>javac addserverintf.java

C:\Users\geeth\Desktop\java programs>start rmiregistry

C:\Users\geeth\Desktop\java programs>java addserver
```

RESULT:

Thus the above program was executed and the output is verified successfully.

EX.NO: 11	**Calculator Application**
DATE:	

AIM:

To Create an applet for a calculator application

ALGORITHM:

Step1: Start the program.

Step2: Create a JFrame, JTextField (non-editable), buttons (0–9, +, −, *, /, ., C, =) and a JPanel.

Step3: Initialize three strings s0, s1, and s2 to store first operand, operator, and second operand.

Step4: Add all buttons and text field to the panel and register action listeners for each button.

Step5: Display the frame with appropriate size and layout.

Step6: When a number or decimal button is pressed, append the value to s0 if operator is empty; otherwise append to s2 and update the text field.

Step7: When an operator (+, −, , /) is pressed, store it in s1; if both operands are present, first evaluate the previous expression and store result in s0.

Step8: When '=' button is pressed, perform the operation based on s1 using s0 and s2,display the result, store result in s0, and clear s1 and s2.

Step9: When 'C' button is pressed, clear s0, s1, and s2, and reset the text field.

Step10: Stop the program when the window is closed.

PROGRAM :

```
import java.awt.event.*;
import javax.swing.*;
import java.awt.*;
class calculator extends JFrame implements ActionListener {
    // create a frame
    static JFrame f;

    // create a textfield
    static JTextField l;

    // store operator and operands
    String s0, s1, s2;

    // default constructor
    calculator()
    {
        s0 = s1 = s2 = "";
    }

    // main function
    public static void main(String args[])
    {
        // create a frame
        f = new JFrame("calculator");

        try {
            // set look and feel
            UIManager.setLookAndFeel(UIManager.getSystemLookAndFeelClassName());
        }
        catch (Exception e) {
            System.err.println(e.getMessage());
        }

        // create a object of class
        calculator c = new calculator();

        // create a textfield
        l = new JTextField(16);

        // set the textfield to non editable
        l.setEditable(false);

        // create number buttons and some operators
        JButton b0, b1, b2, b3, b4, b5, b6, b7, b8, b9, ba, bs, bd, bm, be, beq, beq1;
```

```
// create number buttons
b0 = new JButton("0");
b1 = new JButton("1");
b2 = new JButton("2");
b3 = new JButton("3");
b4 = new JButton("4");
b5 = new JButton("5");
b6 = new JButton("6");
b7 = new JButton("7");
b8 = new JButton("8");
b9 = new JButton("9");

// equals button
beq1 = new JButton("=");

// create operator buttons
ba = new JButton("+");
bs = new JButton("-");
bd = new JButton("/");
bm = new JButton("*");
beq = new JButton("C");

// create . button
be = new JButton(".");

// create a panel
JPanel p = new JPanel();

// add action listeners
bm.addActionListener(c);
bd.addActionListener(c);
bs.addActionListener(c);
ba.addActionListener(c);
b9.addActionListener(c);
b8.addActionListener(c);
b7.addActionListener(c);
b6.addActionListener(c);
b5.addActionListener(c);
b4.addActionListener(c);
b3.addActionListener(c);
b2.addActionListener(c);
b1.addActionListener(c);
b0.addActionListener(c);
be.addActionListener(c);
beq.addActionListener(c);
beq1.addActionListener(c);
```

```
// add elements to panel
        p.add(l);
        p.add(ba);
        p.add(b1);
        p.add(b2);
        p.add(b3);
        p.add(bs);
        p.add(b4);
        p.add(b5);
        p.add(b6);
        p.add(bm);
        p.add(b7);
        p.add(b8);
        p.add(b9);
        p.add(bd);
        p.add(be);
        p.add(b0);
        p.add(beq);
        p.add(beq1);

        // set Background of panel
        p.setBackground(Color.blue);

        // add panel to frame
        f.add(p);

        f.setSize(200, 220);
        f.show();
    }
    public void actionPerformed(ActionEvent e)
    {
        String s = e.getActionCommand();

        // if the value is a number
        if ((s.charAt(0) >= '0' && s.charAt(0) <= '9') || s.charAt(0) == '.') {
            // if operand is present then add to second no
            if (!s1.equals(""))
                s2 = s2 + s;
            else
                s0 = s0 + s;

            // set the value of text
            l.setText(s0 + s1 + s2);
        }
        else if (s.charAt(0) == 'C') {
            // clear the one letter
            s0 = s1 = s2 = "";
```

```
        // set the value of text
        l.setText(s0 + s1 + s2);
    }
    else if (s.charAt(0) == '=') {

        double te;

        // store the value in 1st
        if (s1.equals("+"))
            te = (Double.parseDouble(s0) + Double.parseDouble(s2));
        else if (s1.equals("-"))
            te = (Double.parseDouble(s0) - Double.parseDouble(s2));
        else if (s1.equals("/"))
            te = (Double.parseDouble(s0) / Double.parseDouble(s2));
        else
            te = (Double.parseDouble(s0) * Double.parseDouble(s2));

        // set the value of text
        l.setText(s0 + s1 + s2 + "=" + te);

        // convert it to string
        s0 = Double.toString(te);

        s1 = s2 = "";
    }
    else {
        // if there was no operand
        if (s1.equals("") || s2.equals(""))
            s1 = s;
        // else evaluate
        else {
            double te;

            // store the value in 1st
            if (s1.equals("+"))
                te = (Double.parseDouble(s0) + Double.parseDouble(s2));
            else if (s1.equals("-"))
                te = (Double.parseDouble(s0) - Double.parseDouble(s2));
            else if (s1.equals("/"))
                te = (Double.parseDouble(s0) / Double.parseDouble(s2));
            else
                te = (Double.parseDouble(s0) * Double.parseDouble(s2));

            // convert it to string
            s0 = Double.toString(te);

            // place the operator
            s1 = s;
```

```
            // make the operand blank
            s2 = "";
        }

        // set the value of text
        l.setText(s0 + s1 + s2);
    }
}
```

OUTPUT:

RESULT:

Thus the above program was executed and the output is verified successfully.

EX.NO: 12	Socket Programming
DATE:	

AIM:

Program to send a text message to another system and receive the text message from the system (use socket programming)

ALGORITHM:

Step1: Start the server and create a ServerSocket on port 4999.

Step2: Server waits for a connection using accept() method.

Step3: Start the client and create a Socket to connect to localhost on port 4999.

Step4: Connection is established between client and server.

Step5: Client sends a message ("Is it working") to the server using PrintWriter.

Step6: Server receives the message using BufferedReader and displays it.

Step7: Server sends a response ("yes") back to the client.

Step8: Client receives the response from the server and displays it on the screen.

PROGRAM :

Client1.java:

```
import java.net.*;
import java.io.*;
public class client1{
public static void main(String[] args)throws IOException
{
 Socket s= new Socket("localhost",4999);
 PrintWriter pr=new PrintWriter(s.getOutputStream());
 pr.println("Is it working");
 pr.flush();
 InputStreamReader in=new InputStreamReader(s.getInputStream());
 BufferedReader bf=new BufferedReader(in);
 String str=bf.readLine();
 System.out.println("server : "+str);
}
}
```

Server1.java:

```
import java.net.*;
import java.io.*;
public class server1{
public static void main(String[] args)throws IOException
{
 ServerSocket ss= new ServerSocket(4999);
 Socket s=ss.accept();
  System.out.println("Client conneced");
 InputStreamReader in=new InputStreamReader(s.getInputStream());
 BufferedReader bf=new BufferedReader(in);
 String str=bf.readLine();
 System.out.println("client : "+str);
 PrintWriter pr=new PrintWriter(s.getOutputStream());
 pr.println("yes");
 pr.flush();
}
}
```

OUTPUT:

```
C:\Windows\System32\cmd.e
Microsoft Windows [Version 10.0.22631.5624]
(c) Microsoft Corporation. All rights reserved.

C:\Users\geeth\Desktop\java programs>javac server1.java

C:\Users\geeth\Desktop\java programs>java server1
Client conneced
client : Is it working

C:\Users\geeth\Desktop\java programs>
```

```
C:\Windows\System32\cmd.e
Microsoft Windows [Version 10.0.22631.5624]
(c) Microsoft Corporation. All rights reserved.

C:\Users\geeth\Desktop\java programs>javac client1.java

C:\Users\geeth\Desktop\java programs>java client1
server : yes

C:\Users\geeth\Desktop\java programs>
```

RESULT:

Thus the above program was executed and the output is verified successfully.